Tutorial: Becoming Full-Stack AI Researchers

Sasha Cui[*1], Quan Le[†1], Alexander Mader[‡2], Ping Luo[3], and John Lafferty[4]

[1]Department of Statistics & Data Science, Yale University
[2]Department of Physics, Yale University
[3]Wu Tsai Institute, Yale University
[4]Department of Statistics & Data Science, Yale University

Version: `May 6, 2026`

Abstract

Modern AI research increasingly requires fluency across the full technical stack: data curation, training frameworks, scaling laws, distributed systems, parameter-efficient adaptation, post-training, evaluation, inference, and agentic workflows. This tutorial accompanies the Yale "Becoming Full-Stack AI Researchers" working group and organizes those components into an end-to-end roadmap for researchers who want to move beyond isolated model experiments toward robust AI systems. The paper is paired with a public repository of minimal working examples so that readers can reproduce the core workflows locally, understand the systems tradeoffs involved, and extend the material to their own projects.

Contents

1 Introduction **7**

[*]`sasha.cui@yale.edu`
[†]`quan.le@yale.edu`
[‡]`alexander.mader@yale.edu`

2 PyTorch, JAX, and TensorFlow **9**
2.1 Motivation and Background . . . 9
2.2 Historical Context and Ecosystem . . . 9
2.3 Strengths and Hardware Compatibility . . . 9
2.4 Core Programming Model and Syntax . . . 10
2.5 Ecosystem and Tooling . . . 10
2.6 Installation and Getting Started . . . 11

3 Scaling Laws for Neural Language Models **12**
3.1 Motivation: Predictable Improvement . . . 12
3.2 Power Law Foundations . . . 12
3.2.1 Why Power Laws? . . . 12
3.2.2 Log-Log Linearity . . . 13
3.3 Kaplan Scaling Laws (2020) . . . 13
3.3.1 Key Findings . . . 13
3.4 Chinchilla Scaling Laws (2022) . . . 14
3.4.1 Key Revision . . . 14
3.4.2 Implications . . . 14
3.4.3 Three Scaling Regimes . . . 15
3.5 Practical Applications . . . 15
3.5.1 Model Development Planning . . . 15
3.5.2 Performance Prediction . . . 16
3.5.3 Bottleneck Identification . . . 16
3.6 Data Availability Constraints . . . 16
3.6.1 Internet-Scale Data . . . 16
3.6.2 Synthetic Data . . . 16
3.7 Downstream Task Performance . . . 17
3.7.1 Emergent Abilities . . . 17
3.7.2 Fine-Tuning Efficiency . . . 17
3.8 Inference Scaling . . . 17
3.8.1 Performance vs. Inference Cost . . . 17
3.8.2 Deployment Considerations . . . 18
3.9 Limitations and Future Directions . . . 18
3.9.1 Limitations of Current Scaling Laws . . . 18
3.9.2 Open Questions . . . 18

3.10 Summary 19

4 Data Curation and Dataset Engineering **20**
4.1 Motivation: Data Is a First-Class Research Artifact 20
4.2 Different Kinds of Data Across the Stack 20
4.3 The Core Data Pipeline 20
4.4 Designing Post-Training Data 21
4.5 Contamination, Governance, and Documentation 22
4.6 A Practical Workflow for Research Groups 22
4.7 Summary 23

5 Ray: A Distributed Computing Framework **24**
5.1 Motivation and Background 24
5.2 Historical Context and Ecosystem 24
5.3 Core Programming Model and Syntax 25
5.4 Strengths, Weaknesses, and Integration 26
5.5 Installation and Getting Started 26

6 vLLM and Efficient LLM Serving **28**
6.1 Motivation 28
6.2 Why the KV Cache Matters 28
6.3 The Real Serving Problem: Many Requests at Once 29
6.4 PagedAttention: The Core vLLM Insight 29
6.5 Continuous Batching and Scheduling 30
6.6 Why Researchers Care 30
6.7 Minimal Example 31
6.7.1 Installation 31
6.7.2 Basic Usage 31
6.8 Summary 32

7 DeepSpeed and Memory-Efficient Training **33**
7.1 Motivation: The Training Memory Wall 33
7.2 Why Naive Data Parallelism Is Wasteful 33
7.3 The ZeRO Idea 34
7.4 ZeRO Stages 34
7.5 Why This Matters in Practice 35

7.6 Summary . . . 35

8 Low-Rank Adaptation (LoRA) 36

8.1 Motivation: Why LoRA? . . . 36
8.2 LoRA Mathematical Foundation . . . 36
8.3 LoRA Advantages . . . 36
8.3.1 Multi-tenant Serving . . . 36
8.3.2 Memory Efficiency . . . 37
8.3.3 Compute Efficiency . . . 37
8.4 Can LoRA Match Full Fine-Tuning Performance? . . . 37
8.4.1 The Key Question . . . 37
8.4.2 Conditions for Equal Performance . . . 37
8.5 Experimental Setup and Results . . . 38
8.5.1 Methodology . . . 38
8.5.2 Key Findings . . . 38
8.5.3 Compute Efficiency Analysis . . . 38
8.6 Experimental Results on PBMC3k Dataset . . . 39
8.7 Discussion and Key Takeaways . . . 40
8.7.1 The "Low-Regret Regime" . . . 40
8.7.2 Practical Implications . . . 40
8.8 Best Practices . . . 40
8.9 Conclusion . . . 41

9 Supervised Finetuning 42

9.1 Motivation: The Bridge from Pretraining to Usefulness . . . 42
9.2 Objective and Training Signal . . . 42
9.3 What Good SFT Data Looks Like . . . 43
9.4 Full Finetuning Versus Parameter-Efficient Finetuning . . . 43
9.5 Optimization Choices That Matter . . . 44
9.6 Evaluation After SFT . . . 44
9.7 Common Failure Modes . . . 45
9.8 Practical Recipe . . . 45
9.9 Summary . . . 45

10 LLM Inference and Deployment 46

10.1 Motivation and Background . . . 46

10.2 API-Based Inference . . . 46
10.2.1 OpenRouter: Universal API Gateway . . . 46
10.2.2 Model Selection Criteria . . . 47
10.3 Tool Calling and Function Integration . . . 48
10.3.1 Why Tool Calling Matters . . . 48
10.3.2 Tool Calling Architecture . . . 49
10.3.3 Sequential Tool Calling . . . 49
10.4 Model Context Protocol (MCP) . . . 50
10.4.1 MCP Servers . . . 50
10.5 Prompt Engineering and Optimization . . . 50
10.5.1 OpenAI Model Spec . . . 50
10.5.2 Prompt Optimization with GEPA . . . 51
10.6 Context Window Management . . . 51
10.6.1 Context Length vs. Quality . . . 52
10.6.2 Mitigation Strategies . . . 52
10.7 Practical Considerations . . . 52
10.7.1 Cost Management . . . 52
10.7.2 Latency Optimization . . . 53
10.7.3 Reliability . . . 53
10.8 Summary . . . 53

11 LLM Evaluation and Alignment **54**
11.1 Motivation: Why Evaluation Matters . . . 54
11.2 Evaluation Methodologies . . . 54
11.2.1 Benchmark Design Principles . . . 54
11.2.2 Common Evaluation Paradigms . . . 55
11.3 The LM Evaluation Harness . . . 55
11.3.1 Key Features . . . 56
11.3.2 Usage Pattern . . . 56
11.3.3 Custom Task Definition . . . 56
11.4 Alignment: Beyond Capability . . . 56
11.4.1 Reinforcement Learning from Human Feedback (RLHF) . . . 57
11.4.2 Direct Preference Optimization (DPO) . . . 57
11.4.3 Constitutional AI . . . 58
11.5 Common Evaluation Pitfalls . . . 58

11.5.1 Benchmark Contamination 58
11.5.2 Gaming the Metric 58
11.5.3 Multi-Issue Problems 59
11.5.4 The Alignment Tax 59
11.6 Emerging Evaluation Paradigms 59
11.6.1 Agentic Evaluation 59
11.6.2 Real-World Deployment Metrics 59
11.7 Best Practices 60
11.8 Summary 60

12 Reinforcement Learning for LLM Post-Training 61
12.1 Motivation: When Supervision Is Not Enough 61
12.2 The Basic Objective 61
12.3 The Rollout Loop 61
12.4 Reward Design for Language and Agentic Tasks 62
12.5 Algorithm Families 62
12.6 Systems Considerations 63
12.7 Common Failure Modes 63
12.8 When RL Is Worth It 64
12.9 Summary 64

13 Agentic Systems and Embodied Extensions 65
13.1 From Single Completions to Closed-Loop Systems 65
13.2 Core Components of an Agentic Stack 65
13.3 Tool Use and Protocol Design 65
13.4 Memory, Retrieval, and Working State 66
13.5 Evaluation and Safety for Agents 66
13.6 Multi-Agent and Long-Horizon Patterns 67
13.7 Embodied AI as the Next Extension 67
13.8 Summary 68

14 Limitations, Future Work, and Conclusion 69
14.1 What This Tutorial Covers and What It Does Not 69
14.2 Future Directions 69
14.3 Final Remarks 70

1 Introduction

Large language models (LLMs) and their surrounding ecosystem have evolved rapidly, creating a landscape where the best researchers are expected to navigate multiple layers of the AI stack. From downloading sharded model checkpoints to deploying distributed reinforcement learning pipelines, a modern AI researcher must operate as both an engineer and a scientist.

Problem The traditional division of labor, where some focus only on mathematical modeling and others only on systems, has given way to a full-stack expectation: the ability to move fluidly between data handling, model training, evaluation, deployment, and interpretability. However, we cannot find resources that can introduce AI researchers to understand the entirety of the technical stack.

Contribution In the Fall 2025 semester at Yale University, we organised a **Becoming Full-Stack AI Researchers** working group. This tutorial paper and its accompanying GitHub repository emerged out of the working group. Our goal is not just to introduce individual frameworks in isolation, but to show how they fit together into an end-to-end research workflow. We emphasize hands-on, reproducible, and extensible practices. Each section is written by contributors who prepared minimal working examples (MWEs), demos, slides, and live presentations, thereby capturing both the technical details and the practical challenges encountered by new users.

Application This tutorial can be used as the starting point of a one-semester course on modern AI development as well as for self-study of interested learners who have basic familiarity with Python. Our intended audience includes graduate students, postdoctoral researchers, and faculties who want to broaden their AI engineering capabilities. By the end of this tutorial, readers should be able to replicate common workflows, adapt them to their own research problems, and understand the broader design space of tools and frameworks that constitute modern AI development.

Organization This tutorial is organized into modules covering the full AI development pipeline:

- **Foundations**: PyTorch, JAX, and TensorFlow fundamentals
- **LLMs as Black Boxes**: Inference, evaluation, deployment, and serving

- **Post-Training**: Supervised fine-tuning (SFT) and reinforcement learning (RL)
- **Agentic Systems**: Software and hardware agents, multi-step reasoning
- **Systems**: Distributed training (Ray), efficient serving (vLLM), memory optimization (DeepSpeed)

Each module includes minimal working examples (MWEs), detailed explanations, and practical exercises.

Acknowledgements This course is generously supported by the Wu Tsai Institute at Yale University. We thank Ping Luo, John Lafferty, Linjun Zhang, Anurag Kashyap, Theo Saarinen, and Yuxuan Zhu for helpful comments and suggestions.

2 PyTorch, JAX, and TensorFlow

2.1 Motivation and Background

Modern deep learning frameworks like TensorFlow, PyTorch, and JAX were created with the goals of

1. Efficient tensor (matrix) operations at scale, and
2. Automatic differentiation for optimization algorithms.

Consider a loss function $L(\theta)$ depending on model parameters $\theta \in \mathbb{R}^d$. Training typically happens via the gradient descent algorithm

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t),$$

where η is the learning rate. Computing $\nabla_\theta L(\theta)$ by hand is infeasible for complex neural networks. These frameworks automate it by building a *computational graph* and applying reverse-mode differentiation (backpropagation).

They are written with GPU and TPU in mind for efficient parallel computation so that linera algebra operations and auto diff are executed faster than can be done on CPUs with numpy.

2.2 Historical Context and Ecosystem

In 2015, Google introduced its first large-scale public framework, TensorFlow. In 2018, Google introduced JAX, a functional, composable approach with transformations like `jit`, `grad`, and `pmap`, optimised for TPUs. TensorFlow was once popular but is now on its way to depreciation.

Meanwhile, in 2016, Facebook introduced PyTorch, which became the default for most academic work. Rumours have it that nowadays, Google and xAI work mostly with JAX, OpenAI and Meta/Facebook work mostly with PyTorch, and Anthropic uses both.

2.3 Strengths and Hardware Compatibility

PyTorch. Strengths: Vast ecosystem (Hugging Face, PyTorch Lightning) and greater community support. GPU-first.

JAX. Strengths: clean functional abstractions, XLA compiler, flexibility in creating customised kernels. TPU-first.

2.4 Core Programming Model and Syntax

Here is a simple example written in PyTorch and in Jax. With PyTorch, we can do

```
import torch

x = torch.tensor([2.0], requires_grad=True)
y = x**2 + 3*x + 1
y.backward()   # compute dy/dx
print(x.grad)  # prints tensor([7.])
```

The gradient follows

$$y = x^2 + 3x + 1,$$
$$\frac{dy}{dx} = 2x + 3 = 7 \quad \text{at } x = 2.$$

Equivalently, in JAX, we can do

```
import jax
import jax.numpy as jnp

def f(x):
  return x**2 + 3*x + 1

grad_f = jax.grad(f)
print(grad_f(2.0))  # prints 7.0
```

2.5 Ecosystem and Tooling

Each framework is surrounded by a rich ecosystem of higher-level libraries.

- **PyTorch:** PyTorch Lightning provides structured training loops and experiment organization. The Hugging Face Transformers library, while framework-agnostic, has its deepest integration with PyTorch.

- **JAX:** Flax and Haiku are neural network libraries built specifically for JAX, offering functional-style model definitions.

For distributed and large-scale training, each framework offers its own primitives.

- **PyTorch:** the `torch.distributed` package implements data-parallel and model-parallel training across multiple GPUs and nodes.
- **JAX:** the `pmap` transformation allows parallel execution of functions across multiple devices (TPUs or GPUs), with automatic sharding of arrays.

For the compatibility with the computational hardware,

- **PyTorch:** Built directly on top of CUDA/cuDNN for GPU acceleration.
- **JAX:** Relies on XLA, which in turn generates CUDA kernels when targeting NVIDIA GPUs. Compatible with both GPUs and TUPs.

Finally, orchestration frameworks such as **Ray** integrate with all three, providing a higher-level interface for scaling training jobs beyond a single machine.

2.6 Installation and Getting Started

Installation depends on hardware

- PyTorch: `pip install torch torchvision torchaudio`. Select CUDA version if using GPUs.
- JAX: `pip install jax jaxlib`, or TPU runtime via Colab/Cloud.

Typical pitfalls include

- CUDA version mismatches between PyTorch and system drivers.
- Installing JAX with the wrong CUDA wheel.
- Mixing conda and pip environments without care.

3 Scaling Laws for Neural Language Models

3.1 Motivation: Predictable Improvement

One of the most remarkable discoveries in modern AI is that model performance scales predictably with key factors: model size, dataset size, and compute budget. Understanding these **scaling laws** enables:

- **Resource planning:** Predict performance before training expensive models
- **Optimal allocation:** Balance parameters, data, and compute efficiently
- **Research prioritization:** Identify bottlenecks and high-leverage improvements
- **Capability forecasting:** Anticipate future model capabilities

The power of scaling laws lies in their predictability across orders of magnitude, allowing researchers to extrapolate from smaller experiments to larger deployments.

3.2 Power Law Foundations

Scaling laws typically follow **power law** relationships of the form:

$$L = \frac{\alpha_0}{p^\alpha} \tag{1}$$

where:

- L is the loss (or error metric)
- p is a scaling parameter (e.g., number of parameters, dataset size)
- α_0 is a constant
- α is the scaling exponent

3.2.1 Why Power Laws?

Power laws arise from **scale invariance**: scaling the input p by a factor k produces a proportional response $f(kp) = k^\beta f(p)$. This property necessarily leads to relationships of the form $L = cp^\alpha$.

3.2.2 Log-Log Linearity

Taking logarithms of Equation 1:

$$\log L = \log \alpha_0 - \alpha \log p \tag{2}$$

This linearity in log-log space is the signature of power laws and enables easy fitting and extrapolation.

3.3 Kaplan Scaling Laws (2020)

Kaplan et al. [7] conducted extensive experiments to characterize how language model loss scales with:

- N = Number of model parameters
- D = Dataset size (number of tokens)
- C = Compute budget (FLOPs)

3.3.1 Key Findings

Loss vs. Model Size.

$$L(N) \propto N^{-\alpha_N} \quad \text{where } \alpha_N \approx 0.076 \tag{3}$$

Larger models achieve lower loss, with diminishing returns following a power law.

Loss vs. Dataset Size.

$$L(D) \propto D^{-\alpha_D} \quad \text{where } \alpha_D \approx 0.095 \tag{4}$$

More training data improves performance, but with diminishing returns similar to model size.

Loss vs. Compute.

$$L(C) \propto C^{-\alpha_C} \quad \text{where } \alpha_C \approx 0.050 \tag{5}$$

Given fixed compute, how should one allocate between model size and training tokens?

Optimal Allocation (Kaplan). Kaplan et al. recommended:

- Most compute should go toward larger models
- Dataset size can remain relatively modest
- This led to models like GPT-3 (175B parameters, 300B tokens)

3.4 Chinchilla Scaling Laws (2022)

Hoffmann et al. [4] revisited Kaplan's experiments and found different optimal allocations.

3.4.1 Key Revision

Main Finding: For compute-optimal training, model size (N) and dataset size (D) should scale equally with compute budget (C):

$$N \propto C^{0.50}, \quad D \propto C^{0.50} \tag{6}$$

3.4.2 Implications

Previous Models Were Over-Parameterized. Models like GPT-3 used too many parameters for their training data:

- GPT-3: 175B parameters, 300B tokens
- Chinchilla: 70B parameters, 1.4T tokens
- Result: Chinchilla outperformed GPT-3 despite fewer parameters

Modern Approach. Post-Chinchilla models follow this guidance:

- LLaMA-65B: 65B parameters, 1.4T tokens
- Mistral-7B: 7B parameters, scaled appropriately
- Emphasis on high-quality training data

3.4.3 Three Scaling Regimes

Understanding when different factors dominate:

Small-Scale Regime. Model capacity is the bottleneck:

- Increasing N yields largest improvement
- Limited by model expressiveness
- Example: Sub-billion parameter models

Medium-Scale Regime. Dataset size becomes limiting:

- Need more diverse training data
- Diminishing returns from just scaling N
- Example: 1B–100B parameter range

Large-Scale Regime. Compute efficiency matters most:

- Both N and D must scale together
- Infrastructure and optimization critical
- Example: 100B+ parameter models

3.5 Practical Applications

3.5.1 Model Development Planning

Given compute budget C, determine optimal:

$$N^* = k_1 \cdot C^{0.50} \tag{7}$$

$$D^* = k_2 \cdot C^{0.50} \tag{8}$$

where k_1, k_2 are task-dependent constants.

3.5.2 Performance Prediction

Estimate final loss before training:

$$L_{\text{pred}} = \frac{\alpha_0}{N^{\alpha_N} \cdot D^{\alpha_D}} \tag{9}$$

This enables cost-benefit analysis for model development.

3.5.3 Bottleneck Identification

If current performance is:

- **Far from predicted:** Optimization or data quality issues
- **Close to predicted but insufficient:** Need more N, D, or C
- **Better than predicted:** Architecture or training innovations

3.6 Data Availability Constraints

Scaling laws assume unlimited high-quality data, but reality imposes limits:

3.6.1 Internet-Scale Data

Estimates suggest:

- 10-100 trillion tokens of web text available
- Quality varies dramatically
- Deduplification reduces effective size
- Multilingual data has different coverage

3.6.2 Synthetic Data

When natural data exhausted:

- **Distillation:** Learn from stronger models
- **Self-play:** Generate and curate own data
- **Targeted generation:** Focus on capability gaps
- **Quality over quantity:** Careful curation and filtering

3.7 Downstream Task Performance

Pretraining scaling predicts downstream capabilities:

3.7.1 Emergent Abilities

Some capabilities appear suddenly at scale:

- Multi-step reasoning
- In-context learning
- Code generation
- Multilingual transfer

Debate: Whether these are truly emergent or merely crossing evaluation thresholds.

3.7.2 Fine-Tuning Efficiency

Larger pretrained models typically:

- Require less fine-tuning data
- Converge faster in fine-tuning
- Transfer better across domains
- Benefit more from parameter-efficient methods (LoRA)

3.8 Inference Scaling

Beyond training, inference costs matter:

3.8.1 Performance vs. Inference Cost

Tradeoffs:

- **Larger models:** Better quality, higher latency/cost
- **Smaller models:** Worse quality, lower latency/cost
- **Quantization:** Reduces inference cost at slight quality loss

3.8.2 Deployment Considerations

For production:

$$\text{Total Cost} = \text{Training Cost} + (\text{Inference Cost} \times \text{Number of Requests}) \tag{10}$$

Since models serve billions of requests, inference costs often dominate. This motivates:

- Smaller, cheaper models when sufficient
- Knowledge distillation from large to small models
- Efficient architectures (Mixture-of-Experts)
- Quantization and pruning

3.9 Limitations and Future Directions

3.9.1 Limitations of Current Scaling Laws

- **Architecture dependence:** Scaling exponents may differ for different architectures
- **Task specificity:** Downstream tasks may not scale identically to pretraining loss
- **Data quality:** Scaling laws assume i.i.d. data; quality matters
- **Multimodality:** Vision-language models may follow different laws

3.9.2 Open Questions

- How do Mixture-of-Experts models scale?
- Do different modalities (vision, audio) follow similar laws?
- How does post-training (RL, fine-tuning) scale?
- Are there fundamental limits to scaling?
- How do architectural innovations affect scaling?

3.10 Summary

Key takeaways:

- Model performance scales predictably with size, data, and compute
- Power laws enable performance prediction across orders of magnitude
- Chinchilla scaling: balance parameters and data equally
- Practical implications for model development and deployment
- Consider both training and inference costs
- Data availability and quality impose real constraints

Understanding scaling laws is essential for:

- **Research:** Prioritize high-leverage improvements
- **Development:** Allocate resources optimally
- **Deployment:** Balance quality and cost
- **Strategy:** Anticipate future capabilities

As the field matures, scaling laws will continue evolving, but their fundamental insight—that AI progress is predictable and scalable—remains transformative.

4 Data Curation and Dataset Engineering

4.1 Motivation: Data Is a First-Class Research Artifact

Models are optimized with gradients, but those gradients are entirely determined by the data pipeline. For that reason, modern AI research treats datasets as first-class artifacts rather than as anonymous input files. A strong model pipeline depends not only on architecture and compute, but also on how examples are collected, filtered, deduplicated, documented, and partitioned.

This perspective becomes even more important at scale. As discussed in Section 3, performance is sensitive to both model size and data regime. Once models become large enough, data quality and coverage often dominate naive increases in parameter count.

4.2 Different Kinds of Data Across the Stack

"Data" is not a single category. Different stages of the full-stack workflow rely on different supervision signals:

- **Pretraining corpora:** large heterogeneous text, code, image, or multimodal collections used to learn general representations.
- **Instruction-tuning data:** prompt-response pairs used for supervised finetuning.
- **Preference data:** pairwise or ranked outputs used for alignment and preference optimization.
- **Evaluation data:** held-out tasks, red-team prompts, and benchmark sets used only for measurement.

Conflating these roles is a common mistake. A dataset that is excellent for instruction following may be a poor benchmark, and a benchmark that is useful for evaluation may be too small or too brittle to drive learning.

4.3 The Core Data Pipeline

Most serious AI projects implement some version of the following pipeline:

Acquisition. Collect raw data from web crawls, internal documents, APIs, synthetic generation, or human annotation. At this stage, metadata is often more important than the raw bytes: source, timestamp, license, language, and collection mechanism all affect downstream usability.

Normalization. Convert heterogeneous records into a stable schema. For text data this may include Unicode normalization, HTML stripping, chat template conversion, language identification, or separating prompt fields from target fields.

Deduplication. Repeated examples inflate dataset size without adding information and can distort both training and evaluation. Exact deduplication catches byte-for-byte copies, while fuzzy deduplication catches near-duplicates, templated pages, and repeated boilerplate.

Filtering and quality control. Remove corrupted records, formatting errors, low-information text, unsafe content, and examples that violate the project scope. Quality filters can be rule-based, classifier-based, or model-based. The important point is that the filtering policy itself should be versioned and auditable.

Tokenization and packing. For language models, the data is ultimately consumed as tokens. This stage determines effective sequence length, packing efficiency, and sometimes the number of examples that can fit into a training run. Bad packing decisions can waste a significant fraction of the compute budget.

Mixture design. Large training runs usually combine multiple sources. Mixture weights therefore matter: oversampling code, math, tool traces, or multilingual data can meaningfully change behavior. A smaller but carefully weighted mixture often beats a larger unstructured crawl.

4.4 Designing Post-Training Data

Post-training datasets should be designed around behaviors, not just topics. Useful questions include:

- What should the model do when the answer is ambiguous?
- What response format is required downstream?

- When should the model refuse, defer, or call a tool?
- Which failure cases matter enough to include explicitly?

For instruction tuning, the schema must encode the deployment contract clearly. If the eventual system expects structured tool calls, multi-turn dialogue, or citations, the dataset should demonstrate those behaviors directly. Hoping that the model will infer them from weakly related examples is usually a losing strategy.

Synthetic data can help fill capability gaps, but only if it is curated carefully. Automatically generated data is most effective when it is treated as a candidate pool to be filtered, scored, or adversarially reviewed rather than as free perfect supervision.

4.5 Contamination, Governance, and Documentation

Strong dataset engineering also requires operational discipline.

Train/eval separation. Evaluation leakage is one of the easiest ways to obtain misleading results. Once a benchmark or red-team set influences curation decisions, it is no longer a clean measure of generalization.

Licenses and privacy. A technically impressive dataset may still be unusable if it contains material that cannot legally or ethically be redistributed. Provenance, permissions, and personally identifiable information need to be tracked early, not after the model is already trained.

Versioning. Datasets should be versioned as carefully as code. If a model improves, one should be able to answer whether the change came from better hyperparameters, a different base model, or a revised dataset snapshot.

Documentation. At minimum, each dataset or mixture should document source types, intended use, exclusion criteria, known limitations, and validation checks. Without that documentation, later debugging becomes guesswork.

4.6 A Practical Workflow for Research Groups

For small teams, a pragmatic data workflow is:

1. Start with a narrow target task and define the evaluation set first.

2. Build a small but extremely clean pilot dataset.
3. Validate formatting, tokenization, and splits before scaling collection.
4. Add automatic filters only after manually inspecting enough examples to understand the real failure modes.
5. Keep raw data, cleaned data, and manifests as separate versioned artifacts.
6. Revisit the mixture after the first training run rather than assuming the initial weighting was correct.

This workflow is slower at the beginning, but much faster once experiments become expensive. In practice, many "model" problems turn out to be data pipeline problems in disguise.

4.7 Summary

Dataset engineering is one of the highest-leverage skills in full-stack AI. Good data pipelines create reliable gradients, trustworthy evaluations, and reproducible behavior changes. Poor data pipelines do the opposite, no matter how strong the model or how large the cluster.

5 Ray: A Distributed Computing Framework

5.1 Motivation and Background

As compute and memory requirements of LLMs keep increasing, single-server settings can no longer accommodate their development, training, and serving needs. Ray [9] addresses this challenge by enabling fast and easy application scaling to distributed environments. It provides scalable libraries to support the complete life-cycle of ML applications, pythonic primitives for intuitive development, and integration with existing tools and infrastructure like Kubernetes, AWS, and Azure.

At its core, Ray utilizes three main abstractions:

- **Tasks (remote functions):** A Python function decorated with `@ray.remote`. Each invocation launches a stateless parallel computation executed on any available CPU or GPU.
- **Actors:** A Python class decorated with `@ray.remote` becomes an actor. Unlike tasks, actors are *stateful*—they live on a worker process and can preserve variables across multiple method calls.
- **Worker Node:** Under the hood, Ray maintains a pool of worker nodes, which consist of multiple physical processes. Each node hosts a *raylet* process, which manages the node's shared resources, and one or more worker processes responsible for task submission and execution.

Ray's cluster hosts the head node, which contains the *Global Control Store*(GCS), a server managing cluster-level metadata and operations like scheduling. For fault tolerance, Ray introduced an ownership model, which can be combined with lineage reconstruction to recover lost objects in the event of a node's failure. These components contribute to Ray's core principles for API simplicity and generality, while enabling system performance and reliability. More information about Ray's architecture can be found in the designated architecture whitepaper [15].

5.2 Historical Context and Ecosystem

Researchers at UC Berkeley's RISELab introduced Ray at 2018 [9] and it has since evolved into a widely adopted ecosystem for scaling AI and Python workloads. Today, Ray is maintained by Anyscale and supports a range of libraries for specialized tasks:

- **Ray RLlib**: High-Performance RL
- **Ray Serve**: Scalable & Programmable Model Serving.
- **Ray Tune**: Scalable Hyperparameter Tuning.
- **Ray Train**: Distributed Model Training
- **Ray Data**: Distributed Data Processing

5.3 Core Programming Model and Syntax

The minimal Ray program utilizes standard Python. To enable Ray, we import and initialize it:

```
import ray
ray.init()  # connect to cluster or start locally
```

Remote functions (tasks). Decorate a Python function with `@ray.remote` to execute it as a parallel task:

```
@ray.remote
def square(x):
  return x * x

futures = [square.remote(i) for i in range(4)]
print(ray.get(futures))  # [0, 1, 4, 9]
```

Actors. Use actors to encapsulate state across multiple method calls. An actor is a class decorated with `@ray.remote` and runs persistently on a worker process:

```
@ray.remote
class Counter:
  def __init__(self):
    self.value = 0
  def increment(self):
    self.value += 1
```

```
    return self.value

counter = Counter.remote()
print(ray.get(counter.increment.remote()))  # 1
```

Workers. Behind the scenes, Ray launches a pool of worker processes (Python interpreters). Tasks and actors are scheduled onto these workers by the Ray runtime. While you rarely interact with workers directly, you can control resource allocation (e.g., CPUs, GPUs) when defining tasks or actors:

```
@ray.remote(num_cpus=2)
def heavy_task(x):
  return x ** 2
```

Here, Ray will schedule `heavy_task` only on workers with at least two available CPUs.

5.4 Strengths, Weaknesses, and Integration

Strengths. The Python-first design of Ray lowers the barrier for distributed computing. Its modular libraries (Tune, RLlib, Serve) cover end-to-end machine learning workflows. It integrates naturally with PyTorch, TensorFlow, and JAX.

Weaknesses. Ray introduces runtime overhead for small-scale tasks, and cluster management may require careful tuning for performance. It is best suited for medium to large-scale AI applications in distributed environments rather than fine-grained parallelism in single-server settings.

5.5 Installation and Getting Started

Ray can be installed with:

```
pip install ray
```

Launching Ray locally requires no configuration. To connect to a multi-node cluster, one can start the head node with `ray start --head` and join worker nodes with `ray start --address='...'`.

Ray can also be combined with orchestration systems such as Kubernetes or Slurm for deployment on HPC or cloud environments.

6 vLLM and Efficient LLM Serving

6.1 Motivation

Modern LLM systems are often bottlenecked not by raw model quality but by serving throughput, latency, and memory efficiency. vLLM addresses this systems problem directly. Its goal is to keep GPUs busy while serving many requests with different prompt lengths and different generation horizons. This makes it useful in both deployment settings and research workflows that require large-scale sampling, such as evaluation, synthetic data generation, and reinforcement learning rollouts.

The most important idea behind vLLM is that efficient decoding requires both:

- **fast reuse of past computation**, via the key-value cache; and
- **good memory management**, so those caches can coexist for many concurrent requests.

6.2 Why the KV Cache Matters

The standard scaled dot-product attention operation is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V,$$

where Q, K, and V denote the query, key, and value tensors. During autoregressive decoding, each new token must attend to all previous tokens in the sequence. Without caching, this means repeatedly recomputing keys and values for the same prefix.

The key-value (KV) cache stores those previously computed keys and values so that each decoding step only computes new entries for the newest token. This changes the practical serving story:

- **Without a KV cache:** prefixes are recomputed repeatedly, wasting compute.
- **With a KV cache:** the model reuses prior work and spends compute on the new token only.

Setting	What gets recomputed	Practical consequence
No KV cache	Keys and values for the whole prefix	High latency and poor throughput on long contexts
KV cache enabled	Only the newest keys and values	Much faster incremental decoding

Caching solves the compute problem, but it creates a memory problem: every live request now owns a growing cache in GPU memory.

6.3 The Real Serving Problem: Many Requests at Once

Serving one long sequence is relatively straightforward. Serving thousands of requests is not. Real workloads have three characteristics that make naive implementations inefficient:

- **Asynchronous arrivals:** requests appear at different times.
- **Variable lengths:** prompts and completions have different sizes.
- **Uneven completion times:** some requests finish immediately while others continue decoding.

These properties make static batching inefficient and contiguous memory allocation fragile. A practical serving runtime must handle both scheduling and memory layout at the same time.

6.4 PagedAttention: The Core vLLM Insight

The main systems contribution of vLLM is PagedAttention [8]. Instead of storing each request's KV cache in one large contiguous buffer, vLLM splits the cache into fixed-size pages. A request then owns a list of pages rather than one monolithic block.

This indirection is powerful because it avoids the usual memory-fragmentation problem:

- finished requests can release pages immediately;
- long-running requests can acquire new pages without large copies;

- the runtime can repack work implicitly through page allocation rather than through expensive memory movement.

Conceptually, the runtime maintains a page table that maps logical token positions to physical memory pages. Attention kernels can then read KV values in logical order even if the underlying storage is physically scattered.

6.5 Continuous Batching and Scheduling

Memory efficiency alone is not enough; the scheduler must also keep the GPU occupied. vLLM therefore uses continuous batching. Instead of waiting for a fixed batch to finish, it repeatedly mixes:

- new requests entering the system,
- existing requests that still need another token, and
- requests that can be removed because they have reached a stop condition.

This approach improves utilization because the active batch can change at every decoding iteration. It is especially helpful for bursty real-world workloads and for research jobs that mix short prompts with long completions.

6.6 Why Researchers Care

For full-stack researchers, vLLM is useful in at least four recurring settings:

- **Benchmarking:** evaluating many prompts quickly.
- **Synthetic data generation:** sampling large corpora for SFT or preference data.
- **Agent rollouts:** generating many trajectories for tool-using systems.
- **RL pipelines:** producing rollouts faster so the trainer is not starved for experience.

This is why the repository pairs vLLM material with DeepSpeed and VERL. vLLM handles the inference-heavy side of the loop, while the training stack handles the gradient-heavy side.

6.7 Minimal Example

6.7.1 Installation

```
pip install -U vllm transformers accelerate
```

6.7.2 Basic Usage

```
from vllm import LLM, SamplingParams
import torch

if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
else:
    print("No GPU found, vLLM will be limited")

llm = LLM(
    model="Qwen/Qwen2-1.5B-Instruct",
    trust_remote_code=True,
)

params = SamplingParams(
    temperature=0.7,
    top_p=0.9,
    max_tokens=128,
)

prompts = [
    "Tell me a short story about a robot.",
    "List three benefits of paged KV caching."
]

outputs = llm.generate(prompts, params)

for i, output in enumerate(outputs):
    print(f"Response {i+1}:")
    print(output.outputs[0].text)
```

The accompanying `MWEs/vllm+deepspeed/` notebooks go further and place vLLM in the broader serving and post-training context.

6.8 Summary

vLLM matters because efficient inference is a systems problem, not just a model problem. KV caching reduces redundant computation, PagedAttention reduces memory fragmentation, and continuous batching keeps the hardware busy. Together, these ideas make large-scale LLM serving and rollout generation practical.

7 DeepSpeed and Memory-Efficient Training

7.1 Motivation: The Training Memory Wall

Large-model training is often limited by memory before it is limited by raw compute. A model that fits comfortably for inference may still be impossible to train because the training loop must also store gradients, optimizer states, activations, and temporary communication buffers.

Component	Role during training	Memory intuition
Parameters	Current model weights	About one model copy
Activations	Saved tensors for backpropagation	Scales with depth, batch size, and sequence length
Gradients	Per-parameter derivatives	Roughly one model copy
Optimizer states	Momentum and variance terms for Adam-like optimizers	Often two extra copies
Temporary buffers	Workspace for kernels and communication	Variable overhead

For Adam-style optimizers, the optimizer states alone can dominate the memory budget. This is why naive data parallelism does not solve the real problem: each worker still stores nearly all training states locally.

7.2 Why Naive Data Parallelism Is Wasteful

In ordinary data parallel training, every GPU keeps a full replica of:

- the model parameters,
- the gradients, and
- the optimizer states.

This is easy to reason about and straightforward to implement, but it is memory-inefficient. Adding more GPUs increases aggregate cluster memory, yet it does not proportionally reduce the amount each individual GPU must hold. As a result, some models remain impossible to train even when plenty of total cluster memory exists.

7.3 The ZeRO Idea

DeepSpeed's central idea is ZeRO: the Zero Redundancy Optimizer. Instead of replicating all training states on every device, ZeRO shards those states across the workers and reconstructs what is needed during computation. The high-level principle is simple:

> Do not store the same training state on every GPU unless the algorithm truly requires it.

This idea is powerful because most optimizer states are naturally partitionable. Consider Adam's elementwise update:

$$m_i^t \leftarrow \beta_1 m_i^{t-1} + (1-\beta_1) g_i^t, \tag{11}$$

$$v_i^t \leftarrow \beta_2 v_i^{t-1} + (1-\beta_2)(g_i^t)^2, \tag{12}$$

$$\theta_i^t \leftarrow \theta_i^{t-1} - \alpha \frac{m_i^t/(1-\beta_1^t)}{\sqrt{v_i^t/(1-\beta_2^t)} + \varepsilon}. \tag{13}$$

Each parameter θ_i only needs its local optimizer states and its own gradient. That means the optimizer states can be sharded naturally across devices.

7.4 ZeRO Stages

ZeRO-1: shard optimizer states. Each worker still keeps full parameters and gradients, but only stores optimizer states for a subset of parameters. This removes a large fraction of the memory overhead while requiring relatively modest communication.

ZeRO-2: shard optimizer states and gradients. In naive data parallelism, every worker ends backpropagation with the full gradient tensor. ZeRO-2 replaces the usual all-reduce pattern with reduce-scatter style communication so that the reduced gradients remain partitioned.

- `all-reduce`: every worker receives the full reduced gradient.
- `reduce-scatter`: the reduction is performed, but the result stays sharded.

ZeRO-3: shard parameters as well. The final stage also shards the model parameters themselves. This provides the largest memory savings, but it also introduces

the most communication because parameters must be gathered when layers are used and then released again.

Stage	What is sharded?	Main effect
Naive DP	Nothing	Simple, but memory-heavy
ZeRO-1	Optimizer states	First major memory savings
ZeRO-2	Optimizer states and gradients	Lower backprop footprint
ZeRO-3	Parameters, gradients, and optimizer states	Maximum memory savings

7.5 Why This Matters in Practice

DeepSpeed is useful whenever model size is constrained by training-state memory rather than by pure FLOPs. This is common in:

- large supervised finetuning runs,
- RL post-training where policy updates are expensive,
- experiments on long contexts or large batch sizes, and
- multi-node settings where memory efficiency determines whether a run is feasible at all.

The repository includes a DeepSpeed example in the Ray training folder and a paired serving/training notebook collection in the vLLM+DeepSpeed folder. These materials should therefore be viewed less as isolated demos and more as building blocks for realistic large-model training workflows.

7.6 Summary

DeepSpeed's importance comes from a single systems insight: most of the training memory budget is redundant across workers. ZeRO removes that redundancy step by step. For full-stack AI researchers, understanding this design is essential because many practical training decisions boil down to a tradeoff between memory savings, communication overhead, and implementation complexity.

8 Low-Rank Adaptation (LoRA)

8.1 Motivation: Why LoRA?

The Problem: Modern large language models (LLMs) have trillions of parameters pretrained on massive datasets. Post-training typically uses much smaller datasets focused on specific domains. It seems wasteful to update trillions of parameters for gigabit/megabit training data.
The Solution: Parameter-Efficient Fine-Tuning (PEFT) adjusts large networks by updating only a small subset of parameters [5].
Key Insight: Post-training often requires much less capacity than pretraining, so we can represent updates efficiently with low-rank matrices.

8.2 LoRA Mathematical Foundation

Core Equation: The fundamental LoRA update is expressed as:

$$W' = W + \gamma BA \tag{14}$$

Where:

- $W \in \mathbb{R}^{N \times N}$: Original frozen weight matrix
- $A \in \mathbb{R}^{r \times N}$: Low-rank adapter matrix (input projection)
- $B \in \mathbb{R}^{N \times r}$: Low-rank adapter matrix (output projection)
- $\gamma = \alpha/r$: Scaling factor (maintains learning rate stability across ranks)
- $r \ll N$: Rank of the adaptation (typically 8–64)

Intuition: Instead of updating the full N^2 parameters in W, we learn a low-rank approximation of the update using only $2Nr$ parameters. This is the key insight behind LoRA [6].

8.3 LoRA Advantages

8.3.1 Multi-tenant Serving

LoRA enables efficient multi-tenant serving by keeping original weights unchanged while storing multiple adapters in memory. This allows sampling from different

model versions simultaneously in batched inference. Modern inference engines such as vLLM [8] and SGLang [18] support this natively.

8.3.2 Memory Efficiency

- **Training**: No optimizer state for base weights (often stored in higher precision)
- **Storage**: Adapters are much smaller than full model checkpoints
- **Transfer**: Fast loading/transfer of small adapter files

8.3.3 Compute Efficiency

- **FLOPs per matrix**: Approximately $2N^2 + 6Nr$ vs $3N^2$ for full fine-tuning
- **Ratio**: Approximately 2/3 the compute when $r \ll N$
- **Scaling**: More efficient as model size increases

8.4 Can LoRA Match Full Fine-Tuning Performance?

Answer: Yes, under specific conditions.

8.4.1 The Key Question

Can LoRA match the performance of full fine-tuning, and if so, under which conditions? This question has been comprehensively investigated by Schulman et al. [13].

8.4.2 Conditions for Equal Performance

When LoRA Matches Full Fine-Tuning:

- **Dataset size**: Small-to-medium post-training datasets (typical instruction-tuning/reasoning)
- **Rank**: Sufficient rank to capture essential information
- **Layer coverage**: Apply LoRA to all weight matrices (especially MLP and MoE layers)
- **Hyperparameters**: Proper learning rate scaling with $\gamma = \alpha/r$

When LoRA Underperforms:

- **Capacity exceeded**: When dataset size exceeds LoRA parameter capacity
- **Attention-only**: Applying LoRA only to attention layers (even with matched parameter count)
- **Very large datasets**: Settings resembling pretraining with massive data

8.5 Experimental Setup and Results

8.5.1 Methodology

The experimental methodology follows the approach of Schulman et al. [13]:

- **Models**: Llama 3 series, Qwen3 (including MoE)
- **Datasets**: Tulu3 (instruction-following), OpenThoughts3 (reasoning)
- **Rank sweep**: 1 to 512 across 3 orders of magnitude
- **Learning rate**: Swept for each condition to eliminate LR confounds
- **Metrics**: Log loss (not sampling-based) for clean scaling laws

8.5.2 Key Findings

Learning Curves by Rank: High-rank LoRA ($r = 64$–512) overlaps with full fine-tuning, while low-rank LoRA ($r = 1$–8) underperforms when capacity is exceeded. This pattern is consistent across different model sizes and datasets.

Batch Size Sensitivity: LoRA is less tolerant of large batch sizes than full fine-tuning. This penalty is not mitigated by increasing rank and is a property of the product-of-matrices parametrization.

Layer Coverage Impact: Attention-only LoRA underperforms even with matched parameters. Full coverage (MLP + attention) performs significantly better. MoE layers are particularly important for LoRA effectiveness.

Learning Rate Impact: The optimal learning rate is approximately 10 times higher for LoRA than for full fine-tuning [13].

8.5.3 Compute Efficiency Analysis

- **Theoretical**: LoRA uses approximately 2/3 the FLOPs of full fine-tuning per weight matrix

- **Practical**: Often faster overall due to reduced memory bandwidth
- **Scaling**: Advantage increases with model size

8.6 Experimental Results on PBMC3k Dataset

Our experiments on the PBMC3k single-cell RNA sequencing dataset [17], which contains approximately 3,000 peripheral blood mononuclear cells with gene expression measurements for approximately 20,000 genes, confirm the findings of Schulman et al. [13]:

LoRA Full Coverage Matches Full Fine-Tuning:

- LoRA Full achieved accuracy within 1% of full fine-tuning
- Used only approximately 10% of trainable parameters
- Confirms the main theoretical finding

Attention-Only LoRA Underperforms:

- LoRA Attention-Only achieved lower accuracy than full coverage
- Even with similar parameter counts, layer coverage matters
- Critical insight: Apply LoRA to **all layers**, not just attention

Computational Efficiency:

- **Parameters**: LoRA trains 10× fewer parameters than full fine-tuning
- **Speed**: LoRA is approximately 1.5–2× faster in training time
- **Memory**: Significantly lower memory footprint (no optimizer state for frozen weights)

Hyperparameter Sensitivity:

- **Rank**: Higher ranks ($r = 16, 32$) approach full fine-tuning performance; very low ranks ($r = 2$) show capacity limits
- **Learning Rate**: LoRA is sensitive to learning rate and typically requires 5–10× higher learning rates
- **Batch Size**: Standard batch sizes work well for LoRA

8.7 Discussion and Key Takeaways

8.7.1 The "Low-Regret Regime"

Schulman et al. [13] identify a regime where LoRA performs similarly to full fine-tuning. This regime:

- Covers most post-training scenarios
- Enables efficient fine-tuning in many applications
- Makes powerful adaptation accessible with fewer resources

8.7.2 Practical Implications

- **When to use LoRA**: Most post-training scenarios, especially with limited compute
- **When to use full fine-tuning**: Very large datasets, when maximum performance is critical
- **Best practices**: Apply to all layers, use sufficient rank, proper learning rate scaling

8.8 Best Practices

Based on theoretical understanding and empirical validation:

Implementation Guidelines:

- Apply LoRA to all transformer layers (attention + MLP)
- Start with $r \in [16, 32]$ for medium-sized models
- Set scaling factor $\alpha = 2r$ as default
- Use learning rates 5–10$\times$ higher than for full fine-tuning
- Monitor validation performance to detect capacity limitations

When to Use LoRA:

- Post-training with limited computational resources
- Multi-task serving environments

- Rapid experimentation and iteration
- Storage and transfer constraints

When to Use Full Fine-Tuning:

- Very large datasets approaching pretraining scale
- Maximum performance critical regardless of cost
- Settings where low-rank assumption may not hold

8.9 Conclusion

Low-Rank Adaptation (LoRA) provides an effective and efficient approach to fine-tuning large pretrained models. Our analysis and experiments confirm that:

1. LoRA with full layer coverage matches full fine-tuning performance in the "low-regret regime"
2. Layer coverage is critical—applying LoRA only to attention layers underperforms
3. LoRA achieves 90–99% parameter reduction while maintaining performance
4. Higher learning rates (5–10×) are required compared to full fine-tuning
5. The approach generalizes across domains from NLP to computational biology

LoRA has become a foundational technique in modern deep learning, enabling efficient adaptation of large models while maintaining performance comparable to full fine-tuning.

9 Supervised Finetuning

9.1 Motivation: The Bridge from Pretraining to Usefulness

Pretraining gives a model broad world knowledge, but it does not automatically make the model helpful for a specific downstream task. Supervised finetuning (SFT) is the standard bridge between a pretrained model and a usable assistant, classifier, code model, or domain expert. In the LLM setting, SFT usually means instruction tuning: adapting a base model on curated prompt-response pairs so that it follows the interaction style, task boundaries, and output format that users actually want.

SFT matters because it is often the cheapest stage that produces a large change in behavior. Before reaching for reinforcement learning, researchers should usually ask whether their problem is really missing a reward-learning stage or whether they simply need higher-quality demonstrations, cleaner formatting, or better task decomposition.

9.2 Objective and Training Signal

Given an input prompt x and target completion $y = (y_1, \ldots, y_T)$, the standard SFT objective minimizes the negative log-likelihood of the target tokens:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y)\sim\mathcal{D}}\left[\sum_{t=1}^{T} m_t \log p_\theta(y_t \mid x, y_{<t})\right]. \tag{15}$$

Here m_t is an optional mask indicating which tokens should contribute to the loss. In chat finetuning, this mask is often used to exclude system or user tokens so that the model is trained primarily on assistant responses.

This objective is simple, but the surrounding implementation details matter:

- **Prompt formatting:** The same data can perform very differently under different chat templates.
- **Response boundaries:** End-of-turn markers and stop tokens must match inference-time formatting.
- **Token masking:** Training on the wrong parts of the conversation can distort behavior.
- **Data balance:** Rare but important behaviors are often drowned out by common patterns.

9.3 What Good SFT Data Looks Like

The most common failure mode in SFT is not the optimizer; it is the dataset. High-quality SFT data usually has four properties:

- **Task alignment:** Examples match the tasks the deployment system will actually face.
- **Consistent formatting:** The prompt, tool, and response schema are stable across examples.
- **Answer quality:** Demonstrations are not merely correct; they show the level of detail, caution, and style desired at inference time.
- **Coverage of edge cases:** Refusals, tool errors, ambiguity, and formatting exceptions are represented explicitly.

For instruction tuning, one should think in terms of capabilities rather than raw example counts. A few hundred high-precision examples for an important behavior can be more valuable than tens of thousands of loosely curated pairs. This is especially true when the base model is already strong and the finetuning dataset is intended to reshape behavior rather than teach new world knowledge.

9.4 Full Finetuning Versus Parameter-Efficient Finetuning

SFT can be implemented either by updating all weights or by using a parameter-efficient method such as LoRA from Section 8. The choice depends on data scale, hardware budget, and the amount of behavioral change required.

Full finetuning. This approach gives the most flexibility and can be important when the downstream task is far from the base model's existing distribution. The tradeoff is cost: optimizer states, checkpoints, and distributed training complexity all grow with model size.

Parameter-efficient finetuning. LoRA and related methods are the pragmatic default for many research projects. They reduce memory usage, shorten iteration cycles, and make ablation studies easier to run. For many post-training datasets, this is enough to capture the desired behavior while keeping deployment simple.

9.5 Optimization Choices That Matter

Several decisions repeatedly determine whether an SFT run succeeds:

- **Base model choice:** Start from the smallest model that can plausibly solve the task after adaptation.
- **Sequence packing:** Pack examples efficiently, but do not mix incompatible conversations in ways that corrupt masking.
- **Learning rate:** Strong base models are easy to destabilize; aggressive learning rates can erase useful priors.
- **Batching and accumulation:** Effective batch size matters, especially when instruction data is heterogeneous.
- **Checkpoint selection:** The final step is not always the best checkpoint; evaluate several points during training.

In practice, many teams run a small pilot with a few thousand examples, verify formatting and masking, and only then scale to the full dataset. This catches most infrastructure errors before expensive distributed runs begin.

9.6 Evaluation After SFT

SFT should be evaluated on more than training loss. A useful evaluation stack includes:

- **Held-out task examples:** The primary check for generalization.
- **Format compliance:** Whether outputs satisfy schemas, tool-call syntax, and stop conditions.
- **Behavioral regression tests:** Whether safety, refusal behavior, or style worsened.
- **Human review:** Essential when multiple outputs can be acceptable.

The evaluation section in Section 11 becomes especially important here. SFT often appears successful when judged by loss alone, yet fails on actual task completion, response calibration, or robustness to messy prompts.

9.7 Common Failure Modes

Typical SFT failures include:

- **Template mismatch:** Train-time prompts do not match inference-time prompts.
- **Overfitting to narrow style:** The model becomes verbose, cautious, or repetitive in undesirable ways.
- **Catastrophic forgetting:** Useful general capabilities degrade after a narrow finetuning run.
- **Reward hacking by proxy:** Annotators optimize for superficial patterns that look good in demos but fail in deployment.

These failures are often easier to fix through data redesign than through algorithmic complexity. Better demonstrations, clearer rubrics, and targeted evaluation usually outperform ad hoc optimizer tweaks.

9.8 Practical Recipe

A pragmatic SFT workflow is:

1. Define the target behavior and evaluation set first.
2. Normalize the prompt format, stop tokens, and chat template.
3. Start with a small high-quality dataset and a short pilot run.
4. Decide whether LoRA is sufficient before paying for full finetuning.
5. Evaluate on held-out tasks and inspect failures manually.
6. Expand the dataset only after the pilot exposes the real bottlenecks.

9.9 Summary

Supervised finetuning is the workhorse of modern post-training. It is simple in objective, but demanding in data quality, formatting discipline, and evaluation rigor. For most research groups, mastering SFT provides the fastest path from a generic foundation model to a useful domain-specific system.

10 LLM Inference and Deployment

10.1 Motivation and Background

Inference is the process of using a trained model to generate outputs. While training often receives more attention in the research community, inference is where most of the value accrues in production systems. A single model may be trained once but serve billions of inference requests. Understanding efficient inference, deployment strategies, and optimization techniques is crucial for building practical AI systems.

Modern LLM inference involves several key challenges:

- **Cost management:** API calls can become expensive at scale, requiring careful model selection and optimization.
- **Latency requirements:** Real-time applications demand low-latency responses.
- **Throughput optimization:** Serving many concurrent users efficiently.
- **Capability extension:** Basic LLMs need tools and external knowledge to be truly useful.
- **Context management:** Handling long conversations and large documents.

10.2 API-Based Inference

The simplest way to run inference is through API services. Instead of managing infrastructure, researchers can access powerful models through standardized interfaces.

10.2.1 OpenRouter: Universal API Gateway

OpenRouter [14] unifies multiple model providers into one API service, enabling seamless switching between different models and providers. This abstraction is valuable because:

- **Provider diversity:** Access to major proprietary and open-weight providers through one interface.
- **Automatic failover:** If one provider is down or rate-limited, requests can route to alternatives.

- **Cost optimization:** Easy comparison and switching between providers based on price/performance.
- **Model routing:** Automatic selection of best provider for a given model.

10.2.2 Model Selection Criteria

Choosing the right model involves multiple tradeoffs:

Cost Structure. Providers usually charge separately for input and output tokens, but the cheapest per-token option is not always the cheapest system overall. In practice, compare:

- **Frontier models:** strongest quality, but highest latency and per-token cost.
- **Mid-tier models:** strong quality for most product and research tasks.
- **Small or distilled models:** lowest latency for routing, extraction, and classification.
- **Self-hosted open-weight models:** attractive at high volume, but they introduce infrastructure and operations overhead.

Because pricing changes quickly, always verify the provider's current documentation before budgeting.

Context Length. Different models support different maximum context windows:

- **Short context (32K):** Sufficient for most conversations
- **Medium context (128K):** Handles long documents
- **Long context (1M+):** Entire codebases or book-length documents

Modalities. Models support different input/output types:

- **Text-to-text:** Traditional language models
- **Text+image-to-text:** Vision-language models for image understanding
- **Text+image-to-text+image:** Multimodal systems that can also generate visual outputs

Reasoning Capabilities. Some models support explicit reasoning modes:

- **Standard generation:** Direct token-by-token generation
- **Chain-of-thought:** Step-by-step reasoning in output
- **Reasoning effort control:** Some APIs expose effort or deliberation controls for harder problems

10.3 Tool Calling and Function Integration

Tool calling extends LLM capabilities by allowing models to invoke external functions during generation. This paradigm shift transforms LLMs from pure text generators into orchestrators of complex workflows.

10.3.1 Why Tool Calling Matters

LLMs have inherent limitations:

- **No real-time information:** Training data has a cutoff date
- **No external knowledge:** Cannot access private databases or documents
- **Weak at computation:** Arithmetic and symbolic reasoning are unreliable
- **No actions:** Cannot directly interact with external systems

Tool calling addresses these limitations by giving models the ability to:

- Search the web or databases
- Execute code (Python, SQL, etc.)
- Access APIs (weather, calendar, email)
- Retrieve from knowledge bases
- Perform deterministic computations

10.3.2 Tool Calling Architecture

The typical tool calling flow follows this pattern:

1. **Tool Definition:** Functions are described in a structured schema (JSON):

```
{
  "name": "get_weather",
  "description": "Get current weather for a location",
  "parameters": {
    "type": "object",
    "properties": {
      "location": {"type": "string"},
      "units": {
        "type": "string",
        "enum": ["celsius", "fahrenheit"]
      }
    }
  }
}
```

2. **Model Generation:** The LLM decides whether to call a tool and with what arguments.

3. **Tool Execution:** The application executes the function and returns results.

4. **Result Integration:** The model incorporates tool outputs into its response.

10.3.3 Sequential Tool Calling

Models can make multiple tool calls in sequence to accomplish complex tasks:

```
User: "What's the weather in the capital of France?"
Model: [Calls search("capital of France")]
Tool: "Paris"
Model: [Calls get_weather("Paris")]
Tool: "15°C, partly cloudy"
Model: "The weather in Paris is 15°C and partly cloudy."
```

This capability enables agentic behavior where models can break down complex tasks into multiple steps.

10.4 Model Context Protocol (MCP)

Model Context Protocol [1] standardizes how LLMs interact with external tools and data sources. Unlike ad-hoc tool implementations, MCP provides:

- **Standardized interface:** Consistent API across different tools
- **Server-side execution:** Tools run on dedicated servers
- **Authentication:** Built-in support for secure access
- **Discovery:** Automatic tool schema detection

10.4.1 MCP Servers

Popular MCP servers include:

- **Notion:** Access to workspace documents and databases
- **Google Calendar:** Event management and scheduling
- **GitHub:** Repository access and code search
- **Slack:** Team communication integration
- **Custom servers:** Organizations can build domain-specific MCP servers

10.5 Prompt Engineering and Optimization

The inputs provided to LLMs dramatically affect output quality. Prompt engineering is the practice of carefully designing these inputs to achieve desired behaviors.

10.5.1 OpenAI Model Spec

OpenAI's Model Spec [10] defines a chain of command for instructions:

1. **Root:** Model Spec root sections (unchangeable)
2. **System:** Model Spec system sections and system messages (OpenAI-controlled)

3. **Developer:** Developer messages and instructions (application-level)
4. **User:** User messages (end-user input)
5. **Guideline:** Model Spec guideline sections (suggestions)

Developer-level instructions (often called "system prompts") allow applications to guide model behavior while respecting safety constraints.

10.5.2 Prompt Optimization with GEPA

Gradient-free Prompt Evolution Algorithm (GEPA) [16] automates prompt optimization:

- **Iterative improvement:** Evolves prompts through multiple generations
- **Metric-driven:** Optimizes for user-defined success criteria
- **Reflection:** Uses model self-evaluation to guide improvements
- **No gradients:** Works with any LLM API

The typical GEPA workflow:

1. Define evaluation metric (e.g., accuracy on test set)
2. Start with baseline prompt
3. Model reflects on failures and suggests improvements
4. Test improved prompts
5. Iterate until convergence

In experiments, GEPA can improve prompt performance by 10–40% on challenging tasks like mathematical reasoning.

10.6 Context Window Management

Long context windows enable powerful applications but introduce challenges:

10.6.1 Context Length vs. Quality

Models exhibit performance degradation with very long contexts:

- **Lost in the middle:** Information in the middle of long contexts is harder to retrieve
- **Attention dilution:** With limited attention capacity, relevant information gets less focus
- **Context rot:** Performance degrades as context approaches maximum length

10.6.2 Mitigation Strategies

Several techniques address context limitations:

- **Conversation summarization:** Compress long histories into concise summaries
- **Retrieval augmentation:** Fetch only relevant context on-demand
- **Hierarchical processing:** Process documents in chunks with aggregation
- **Attention steering:** Explicitly guide model focus to important sections

10.7 Practical Considerations

10.7.1 Cost Management

Inference costs can scale rapidly. Key strategies:

- **Model routing:** Use cheaper models for simple queries, expensive ones for hard problems
- **Caching:** Store and reuse responses for common queries
- **Batching:** Group multiple requests for efficiency
- **Output length limits:** Constrain max_tokens to control costs

10.7.2 Latency Optimization

For real-time applications:

- **Streaming:** Display tokens as they're generated
- **Speculative decoding:** Generate multiple candidates in parallel
- **Model selection:** Use smaller, faster models when appropriate
- **Regional providers:** Choose geographically close API endpoints

10.7.3 Reliability

Production systems need robust error handling:

- **Rate limit handling:** Implement exponential backoff
- **Failover:** Route to alternative providers when needed
- **Validation:** Check output format and content
- **Monitoring:** Track costs, latency, and error rates

10.8 Summary

Effective LLM inference requires understanding:

- Model selection tradeoffs (cost, performance, capabilities)
- Tool calling for extending capabilities
- Prompt engineering for optimal outputs
- Context management for long conversations
- Practical considerations for production deployment

The inference ecosystem continues to evolve rapidly, with new models, techniques, and best practices emerging regularly. The skills covered in this section provide a foundation for building robust, cost-effective LLM applications.

11 LLM Evaluation and Alignment

11.1 Motivation: Why Evaluation Matters

As language models grow in capability, rigorous evaluation becomes essential for:

- **Progress measurement:** Tracking improvements across model versions
- **Model selection:** Choosing appropriate models for specific tasks
- **Research validation:** Demonstrating the effectiveness of new techniques
- **Safety assessment:** Identifying potential harms and failure modes
- **Capability mapping:** Understanding what models can and cannot do

However, LLM evaluation presents unique challenges:

- **Open-ended generation:** Many valid responses to the same prompt
- **Capability breadth:** Models span reasoning, coding, creativity, and more
- **Benchmark contamination:** Models may have seen evaluation data during training
- **Gaming metrics:** Optimizing for benchmarks without genuine improvement
- **Alignment vs. capability:** Tradeoffs between helpfulness, honesty, and harmlessness

11.2 Evaluation Methodologies

11.2.1 Benchmark Design Principles

Effective benchmarks share common characteristics:

- **Diverse tasks:** Cover multiple domains and capabilities
- **Difficulty range:** Include both easy and hard examples
- **Clear metrics:** Objective, reproducible scoring
- **Low contamination:** Minimize training data overlap
- **Regular updates:** Refresh to avoid overfitting

11.2.2 Common Evaluation Paradigms

Multiple Choice. Present options and measure selection accuracy:

- **Advantages:** Objective scoring, easy to scale
- **Disadvantages:** May not reflect real usage, susceptible to guessing
- **Examples:** MMLU, HellaSwag, ARC

Exact Match. Compare generated text to reference answers:

- **Advantages:** Clear success criterion
- **Disadvantages:** Inflexible, penalizes valid alternatives
- **Examples:** GSM8K (math), HumanEval (code)

Human Evaluation. Expert judgment of outputs:

- **Advantages:** Captures nuanced quality
- **Disadvantages:** Expensive, slow, subjective
- **Examples:** Chatbot Arena, LMSYS leaderboard

LLM-as-Judge. Use strong models to evaluate weaker ones:

- **Advantages:** Scalable, captures complex criteria
- **Disadvantages:** Inherits judge model biases
- **Examples:** AlpacaEval, MT-Bench

11.3 The LM Evaluation Harness

The `lm-evaluation-harness` [3] from EleutherAI provides a unified framework for evaluating language models across dozens of benchmarks.

11.3.1 Key Features

- **Standardized interface:** Consistent API across 200+ tasks
- **Reproducibility:** Fixed random seeds and evaluation protocols
- **Efficiency:** Batched inference and caching
- **Extensibility:** Easy to add custom tasks
- **Model support:** HuggingFace, vLLM, OpenAI API, and more

11.3.2 Usage Pattern

A typical evaluation workflow:

```
lm_eval --model hf \
  --model_args pretrained=meta-llama/Llama-3.1-8B \
  --tasks mmlu,gsm8k,hellaswag \
  --batch_size 8 \
  --output_path results/
```

This produces a detailed report with per-task metrics, aggregate scores, and statistical confidence intervals.

11.3.3 Custom Task Definition

Users can define new evaluation tasks in YAML:

```
task: my_custom_task
dataset_name: my_org/my_dataset
output_type: multiple_choice
metric: acc
num_fewshot: 5
```

This flexibility enables evaluation on proprietary or domain-specific benchmarks.

11.4 Alignment: Beyond Capability

Alignment ensures models are helpful, honest, and harmless—not just capable.

11.4.1 Reinforcement Learning from Human Feedback (RLHF)

RLHF [11] trains models to generate outputs humans prefer:

Phase 1: Supervised Fine-Tuning (SFT). Train on high-quality demonstration data:

- Collect expert demonstrations
- Fine-tune base model to imitate
- Produces helpful but not necessarily preferred outputs

Phase 2: Reward Model Training. Learn a preference model from comparisons:

- Present pairs of model outputs
- Humans indicate which is better
- Train classifier to predict preferences
- Reward model scores: $r_\theta(x, y)$

Phase 3: RL Policy Training. Optimize policy to maximize reward:

$$\max_{\pi} \mathbb{E}_{x\sim\mathcal{D}, y\sim\pi(\cdot|x)}[r_\theta(x, y) - \beta \mathrm{KL}(\pi \| \pi_{\mathrm{ref}})] \tag{16}$$

The KL penalty prevents the policy from deviating too far from the reference model, avoiding reward hacking.

11.4.2 Direct Preference Optimization (DPO)

DPO [12] simplifies RLHF by directly optimizing on preference data:

- **No reward model:** Directly use preference pairs
- **Simpler training:** Standard supervised learning
- **Better stability:** Avoids RL optimization challenges

The DPO loss:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right] \tag{17}$$

where y_w is the preferred completion and y_l is the dispreferred one.

11.4.3 Constitutional AI

Constitutional AI [2] encodes explicit principles:

- **Self-critique:** Model critiques its own outputs
- **Self-revision:** Generates improved versions
- **Preference learning:** Learns from self-comparisons
- **Transparency:** Explicit ethical guidelines

This approach reduces reliance on human labels while maintaining alignment.

11.5 Common Evaluation Pitfalls

11.5.1 Benchmark Contamination

Models may have seen evaluation data during pretraining:

- **Detection:** Check for memorization with perturbed examples
- **Mitigation:** Use private test sets, regular benchmark rotation
- **Reporting:** Disclose known overlaps transparently

11.5.2 Gaming the Metric

Optimizing for benchmarks can degrade real-world performance:

- **Goodhart's Law:** "When a measure becomes a target, it ceases to be a good measure"
- **Solution:** Evaluate on diverse, held-out tasks
- **Holistic assessment:** Combine quantitative and qualitative evaluation

11.5.3 Multi-Issue Problems

A single output may fail in multiple ways:

- Incorrect reasoning *and* poor formatting
- Helpful content *but* potential harm
- Correct answer *but* inefficient approach

Effective evaluation must disentangle these dimensions.

11.5.4 The Alignment Tax

Alignment techniques may reduce capability:

- **Observation:** Aligned models sometimes score lower on capability benchmarks
- **Explanation:** Safety constraints limit certain behaviors
- **Debate:** Whether this represents genuine capability loss or appropriate restraint

Research continues on alignment methods that preserve or enhance capabilities.

11.6 Emerging Evaluation Paradigms

11.6.1 Agentic Evaluation

As models become more agentic, evaluation must adapt:

- **Multi-step tasks:** Measure planning and execution
- **Tool use:** Evaluate when and how models call tools
- **Long-horizon tasks:** Track performance over extended interactions
- **Failure recovery:** Assess adaptation when plans go wrong

11.6.2 Real-World Deployment Metrics

Beyond benchmarks, track production performance:

- **User satisfaction:** Thumbs up/down, engagement time
- **Task success rate:** Did the user accomplish their goal?

- **Safety incidents:** Harmful outputs, jailbreaks
- **Cost efficiency:** Quality per dollar spent

11.7 Best Practices

For Researchers:

- Report results on diverse benchmarks
- Include statistical significance tests
- Disclose evaluation protocols completely
- Consider both capability and alignment
- Test for benchmark contamination

For Practitioners:

- Evaluate on your specific use cases
- Combine automated and human evaluation
- Monitor performance over time
- A/B test model changes
- Track both quality and cost metrics

11.8 Summary

Effective LLM evaluation requires:

- Understanding evaluation paradigms and their tradeoffs
- Using standardized tools like `lm-eval-harness`
- Considering alignment alongside capability
- Avoiding common pitfalls (contamination, gaming)
- Adapting to emerging evaluation needs (agentic systems)

As models become more capable, evaluation methodologies must evolve to capture new dimensions of performance, safety, and alignment.

12 Reinforcement Learning for LLM Post-Training

12.1 Motivation: When Supervision Is Not Enough

Supervised finetuning works well when high-quality target outputs are available, but many important behaviors are not naturally specified by a single reference answer. Tool use, long-horizon planning, interaction quality, and strategic reasoning often depend on delayed or partially observed feedback. Reinforcement learning (RL) becomes useful in precisely these settings because it allows the model to improve from scalar or comparative signals rather than from fully written demonstrations.

In language-model post-training, RL is best viewed as a way to optimize behavior after SFT has already produced a competent base policy. The model starts from an instruction-following regime and is then nudged toward outcomes that are preferred, more effective, or more robust under interaction.

12.2 The Basic Objective

Let $\pi_\theta(y \mid x)$ denote a policy that generates a completion y for prompt x. A common KL-regularized objective is

$$\max_{\pi_\theta} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)} \left[r(x,y) - \beta \, \mathrm{KL}(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid x)) \right], \tag{18}$$

where $r(x,y)$ is the reward and π_{ref} is a reference model, often the SFT checkpoint. The KL term prevents the optimized policy from drifting too far from the reference behavior in pursuit of reward.

This formulation captures the core idea behind RLHF [11]: learn from outcomes while retaining the competence and stability of the supervised model that came before.

12.3 The Rollout Loop

Practical RL for LLMs usually alternates between four stages:

1. Sample prompts from a dataset or environment.
2. Generate rollouts with the current policy.
3. Score those rollouts with rewards, preferences, or environment feedback.
4. Update the policy using the collected trajectories.

The key engineering fact is that rollout generation is often the dominant systems cost. This is why the stack discussed earlier in the tutorial matters: fast inference engines help with sampling, distributed trainers help with the update step, and orchestration frameworks coordinate the whole loop.

12.4 Reward Design for Language and Agentic Tasks

Reward design is the central research problem in RL post-training. For LLMs, rewards often come from one or more of the following sources:

- **Outcome rewards:** Did the final answer solve the task?
- **Preference rewards:** Which of two completions do humans or a reward model prefer?
- **Process rewards:** Were intermediate reasoning steps, tool calls, or plans useful?
- **Environment rewards:** Did the agent actually complete an action sequence in a simulator or software environment?

The accompanying `MWEs/agentic_rl_workshop.ipynb` notebook emphasizes a useful distinction between outcome rewards and process rewards. Outcome rewards are sparse but aligned with end goals. Process rewards are denser and can accelerate learning, especially for tool use, action formatting, and multi-step reasoning. The tradeoff is that badly designed process rewards can teach the model to look busy rather than to succeed.

12.5 Algorithm Families

Online RL. Methods such as PPO remain conceptually important because they optimize the policy on trajectories sampled from the current model. This makes them suitable when exploration matters or when behavior depends on multi-step interaction with tools or environments.

Preference optimization. Methods such as DPO [12] use preference pairs directly and avoid an explicit online rollout loop during optimization. They are often simpler and more stable than full RL, which is why many practical alignment pipelines now use them as a default starting point.

Hybrid pipelines. In practice, teams often combine the two. A model may receive SFT first, then preference optimization, and only then a more expensive online RL stage if interaction quality still needs improvement.

12.6 Systems Considerations

Full-stack thinking is especially important for RL because the algorithm is inseparable from the infrastructure. A typical modern pipeline looks like this:

- **Inference engine:** vLLM or a similar fast decoder for trajectory generation.
- **Trainer:** DeepSpeed or another distributed training backend for large-policy updates.
- **Orchestration:** Ray-style scheduling when rollout workers, reward workers, and trainers must run concurrently across multiple machines.
- **Workflow layer:** Containerized or framework-specific stacks such as the repository's `MWEs/verl` workflow for end-to-end PPO-style experiments.

This is why reinforcement learning is rarely just an "algorithm section" in practice. It is a systems problem with an optimization loop inside it.

12.7 Common Failure Modes

Typical RL failures for LLMs include:

- **Reward hacking:** The policy exploits weaknesses in the reward instead of learning the intended behavior.
- **Mode collapse:** The model becomes repetitive or narrow because the reward over-favors a small set of outputs.
- **Over-optimization:** Reward improves while actual task quality or human preference worsens.
- **Systems bottlenecks:** Sampling or data movement dominates the wall-clock budget, making the training loop impractical.

These issues make offline evaluation and ablation crucial. A reward function should be stress-tested before a large RL run, not after it has consumed the compute budget.

12.8 When RL Is Worth It

RL is usually worth the complexity when:

- the task requires interaction rather than single-shot completion;
- the best feedback is scalar, comparative, or delayed;
- the model must learn tool strategy rather than just tool syntax;
- the intended behavior depends on long-horizon tradeoffs.

If those conditions do not hold, better SFT data or better evaluation may be a cheaper and more reliable intervention.

12.9 Summary

Reinforcement learning extends post-training beyond fixed demonstrations. It is most powerful when the problem involves delayed feedback, multi-step interaction, or rewardable behavior that is hard to express as a single gold answer. For full-stack AI researchers, the important lesson is that successful RL depends as much on reward design and systems throughput as on the update rule itself.

13 Agentic Systems and Embodied Extensions

13.1 From Single Completions to Closed-Loop Systems

An LLM becomes an agent when it is placed inside a loop that observes an environment, chooses actions, receives feedback, and updates its internal state or external context over time. This is a shift in problem formulation. The output is no longer just a response string; it is part of a control policy operating over tools, files, browsers, APIs, or even physical robots.

The practical consequence is that model quality alone is no longer sufficient. Agent performance depends on tool interfaces, state management, failure recovery, permissions, and evaluation methodology just as much as on the underlying checkpoint.

13.2 Core Components of an Agentic Stack

Most agent systems contain the following components:

- **Policy or planner:** the model that proposes actions or subgoals.
- **Tool layer:** functions, APIs, browsers, code execution, or databases the model can use.
- **State and memory:** conversation history, retrieved documents, working notes, or structured task state.
- **Environment:** the external world in which actions have consequences.
- **Monitor or critic:** logic for validation, rollback, retries, or escalation.

Researchers often focus on the first item because it is the most visible, but many production failures originate in the others. An excellent planner with weak tooling and poor validation is still a weak agent.

13.3 Tool Use and Protocol Design

Tool use is the most direct way to extend a model beyond static next-token prediction. The model can search, run code, read private documents, or act on external systems, but only if the interface is explicit enough for the model to use reliably and safe enough for the application to execute.

This is where structured schemas and interoperability matter. The Model Context Protocol (MCP) [1] is one example of an emerging standard for exposing tools and data sources through consistent interfaces. Regardless of the specific protocol, good tool design usually requires:

- **Clear argument schemas:** the model should know exactly what fields to supply.
- **Deterministic outputs when possible:** stable tool behavior simplifies debugging.
- **Permission boundaries:** some actions must require confirmation or sandboxing.
- **Machine-readable errors:** agents need structured failure modes, not vague strings.

13.4 Memory, Retrieval, and Working State

Agentic tasks quickly exceed the usefulness of raw conversation history. Effective systems therefore distinguish between several kinds of context:

- **Short-term context:** the immediate dialogue and tool results.
- **Retrieved context:** documents or facts fetched on demand.
- **Working memory:** scratchpads, plans, or structured intermediate state.
- **Long-term memory:** durable user preferences, project metadata, or prior outcomes.

This separation matters because not every piece of information should stay in the prompt forever. Good agents decide what to keep, what to summarize, what to retrieve again, and what to discard.

13.5 Evaluation and Safety for Agents

Agent evaluation is harder than single-turn benchmark evaluation because failures can happen at many layers:

- the plan may be wrong;

- the tool choice may be wrong;
- the tool call may be malformed;
- the recovery strategy may fail after an error;
- the final answer may hide earlier mistakes.

As a result, agent evaluation should include trajectory-level metrics such as task completion rate, number of tool calls, recovery after failure, and cost per successful task. Safety evaluation should also check whether the agent respects permission boundaries, avoids unsafe actions, and handles ambiguous instructions conservatively.

13.6 Multi-Agent and Long-Horizon Patterns

Some tasks are easier when the system is decomposed into roles rather than solved by a single monolithic prompt. Common patterns include:

- **Planner-executor:** one model decomposes the task and another carries out steps.
- **Actor-critic:** one model proposes actions and another critiques them.
- **Specialist routing:** different models or tools handle code, retrieval, planning, or summarization.

These patterns are useful only when the coordination cost is lower than the quality gain. More agents do not automatically mean better performance.

13.7 Embodied AI as the Next Extension

The repository extends the agentic story into robotics through `MWEs/Robotics`. This is a natural continuation of the same full-stack perspective. In robotics, the "tools" are motors, controllers, cameras, and simulators; the "environment" is physical; and the cost of mistakes is much higher.

Three practical lessons carry over from software agents to embodied agents:

- **Interfaces matter:** observation and action schemas shape what the policy can learn.
- **Evaluation matters:** simulation, real-world deployment, and benchmark design rarely align perfectly.

- **Data matters:** trajectory quality, teleoperation data, and task coverage strongly affect policy performance.

Frameworks such as Robosuite, RoboVerse, and LeRobot illustrate different points in this design space: simulator-focused research, multi-simulator benchmarking, and data-centric real-robot workflows. For full-stack researchers, the main insight is that agentic ideas do not stop at text interfaces; they generalize to physical systems with the same need for robust tooling, observability, and feedback loops.

13.8 Summary

Agentic systems are best understood as model-centered software stacks rather than as prompts with extra steps. They combine inference, tools, memory, evaluation, and environment interaction into a closed loop. Learning to reason about that loop is one of the clearest markers of becoming a full-stack AI researcher.

14 Limitations, Future Work, and Conclusion

14.1 What This Tutorial Covers and What It Does Not

This tutorial is intentionally broad. Its purpose is to help researchers see how the major components of modern AI systems fit together, not to replace the official documentation for every framework discussed. As a result, several important topics are only introduced at a high level:

- **Cluster operations:** Production-grade observability, autoscaling, and incident response are deep disciplines in their own right.
- **Model internals:** We emphasize workflow and systems integration more than architectural novelty.
- **Safety and governance:** We discuss evaluation and guardrails, but organizational policy and legal review require additional expertise.
- **Moving targets:** APIs, model availability, and recommended infrastructure evolve quickly.

Readers should therefore treat the manuscript as a roadmap and the repository as a launch point, then move to framework-specific references once they know which layer of the stack they need to go deeper on.

14.2 Future Directions

The full-stack AI workflow will likely continue shifting in three directions.

First, stronger post-training loops. The boundary between SFT, preference optimization, and online reinforcement learning is becoming more fluid. Researchers increasingly need to reason about data collection, reward design, and deployment feedback as one continuous loop rather than isolated stages.

Second, more agentic systems. Inference is no longer just single-turn text generation. Tool use, memory, planning, evaluation, and execution environments are becoming first-class concerns. As a result, the practical unit of AI research is moving from the model checkpoint to the whole system wrapped around it.

Third, tighter integration between software and hardware agents. The same ideas that matter for tool-using assistants—structured observations, action spaces, rollback, safety constraints, and reward signals—also appear in robotics and embodied AI. We expect the distinction between language-only agents and embodied agents to narrow over time.

14.3 Final Remarks

Becoming a full-stack AI researcher does not mean mastering every tool equally. It means building enough fluency across the stack to diagnose bottlenecks correctly, choose the right abstraction for the problem at hand, and move between modeling, data, systems, and evaluation without losing the thread of the research question.

That is the central message of this tutorial. PyTorch and JAX teach how models are expressed, scaling laws and data work explain why models improve, Ray/vLLM/DeepSpeed show how systems scale, LoRA and SFT make adaptation practical, evaluation and RL sharpen the learning signal, and agentic workflows connect models to real environments. The accompanying repository is designed to make those connections concrete through minimal working examples.

The field will keep changing, but the underlying habit of mind remains stable: strong AI research now requires both scientific judgment and engineering range. Developing that range is the point of becoming full-stack.

References

[1] Anthropic. *Model Context Protocol*. `https://modelcontextprotocol.io`. 2024.

[2] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, et al. "Constitutional AI: Harmlessness from AI feedback". In: *arXiv preprint arXiv:2212.08073* (2022).

[3] EleutherAI. *Language Model Evaluation Harness*. `https://github.com/EleutherAI/lm-evaluation-harness`. 2024.

[4] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. "Training compute-optimal large language models". In: *arXiv preprint arXiv:2203.15556* (2022).

[5] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, et al. "Parameter-efficient transfer learning for NLP". In: *International Conference on Machine Learning* (2019), pp. 2790–2799.

[6] Edward J Hu, Yelong Shen, Phillip Wallis, et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *International Conference on Learning Representations* (2022).

[7] Jared Kaplan, Sam McCandlish, Tom Henighan, et al. “Scaling laws for neural language models”. In: *arXiv preprint arXiv:2001.08361* (2020).

[8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al. “Efficient Memory Management for Large Language Model Serving with PagedAttention”. In: *Symposium on Operating Systems Principles* (2023).

[9] Philipp Moritz, Robert Nishihara, Stephanie Wang, et al. “Ray: a distributed framework for emerging AI applications”. In: OSDI’18. Carlsbad, CA, USA: USENIX Association, 2018, pp. 561–577. ISBN: 9781931971478.

[10] OpenAI. *Model Spec: Desired Behavior for OpenAI Models*. `https://cdn.openai.com/spec/model-spec-2024-05-08.html`. 2024.

[11] Long Ouyang, Jeffrey Wu, Xu Jiang, et al. “Training language models to follow instructions with human feedback”. In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 27730–27744.

[12] Rafael Rafailov, Archit Sharma, Eric Mitchell, et al. “Direct preference optimization: Your language model is secretly a reward model”. In: *arXiv preprint arXiv:2305.18290* (2023).

[13] Dan Schulman, Apoorv Vyas, Yuhui Li, et al. “LoRA Learns Less and Forgets Less”. In: *arXiv preprint arXiv:2405.09673* (2024).

[14] OpenRouter Team. *OpenRouter: Unified LLM API*. `https://openrouter.ai`. 2024.

[15] Ray Team. *Ray v2 Architecture Whitepaper*. Available online: Ray Architecture Whitepaper. (Visited on 10/21/2025).

[16] Yuxuan Zhang et al. “Gradient-free Prompt Evolution Algorithm for Large Language Models”. In: *arXiv preprint arXiv:2507.19457* (2025).

[17] Grace XY Zheng, Jessica M Terry, Phillip Belgrader, et al. “Massively parallel digital transcriptional profiling of single cells”. In: *Nature Communications* 8.1 (2017), p. 14049.

[18] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, et al. "Efficiently Programming Large Language Models using SGLang". In: *arXiv preprint arXiv:2312.07104* (2023).

Acknowledgments

We thank the contributors to the Yale working group and the accompanying repository for helping shape the tutorial materials. We are especially grateful to Ping Luo, John Lafferty, Linjun Zhang, Anurag Kashyap, Theo Saarinen, and Yuxuan Zhu for feedback on the course content and to the Wu Tsai Institute for supporting the broader effort.